DWMusicBrainz · Bases de Datos Avanzadas

# Documentación técnica de DWMusicBrainz

Qué se hizo, con qué tecnologías y por qué
Migración a Databricks con 58.809.976 pistas

Juan Sebastián Rincón Farelo
Integrantes: Juan Rincón, Sebastián Russo, Jesús Figueroa
Docente: Amilkar Sierra Romano
Ingeniería de Sistemas · Universidad Popular del Cesar
Valledupar, noviembre de 2026

# Índice


**Resumen para la defensa** 4

Las cinco ideas que debe recordar . . . 4

Cómo usar este documento . . . 4

**Requisito y punto de partida** 5

Recursos disponibles . . . 5

Decisiones que tomó el usuario . . . 5

**Cronología del proceso** 6

Prueba de capacidad . . . 6

**Arquitectura** 7

Tareas del job . . . 7

**Modelo de datos** 8

Decisión de grano . . . 8

Conteos por tabla . . . 8

Reglas de negocio del modelo . . . 9

**Reglas de calidad** 10

Reglas EDA heredadas . . . 10

**Decisiones de tecnología** 11

ADR-01. Plataforma . . . 11

ADR-02. Plan de Databricks . . . 12

ADR-03. Lenguaje . . . 12

ADR-04. Formato de tablas . . . 12

ADR-05. Archivos crudos . . . 13

ADR-06. Orquestación . . . 13

ADR-07. Organización física . . . 13

ADR-08. Cómputo . . . 14

ADR-09. Visualización . . . 14

ADR-10. Cálculo de primeras pistas . . . 14

ADR-11. Carga de archivos . . . 15

ADR-12. Etiquetas de dimensiones . . . 15

**Errores encontrados y correcciones** 16

Lección . . . 16

**Evidencia de validación** 17

Pruebas . . . 17

Reconciliación . . . 17
Corridas . . . 17
Filtros del dashboard contra SQL . . . 17
**Resultados** **18**
**Limitaciones conocidas** **19**
**Guía de defensa** **20**
Preguntas difíciles . . . 20
Modelo y datos . . . 20
Tecnología . . . 21
Resultados . . . 21
**Reproducción y mantenimiento** **22**
Archivos principales . . . 22
**Referencias** **23**

Sección 1

# Resumen para la defensa

**El proyecto cumple el requisito de más de 50 millones de registros.** DWMusicBrainz pasó de PostgreSQL a Databricks y ahora procesa el volcado completo de MusicBrainz: **58.809.976 pistas** en `gold.fact_pista`, con un pipeline de 633 segundos y 116 de 116 reglas bloqueantes aprobadas.

| | | | |
|---|---|---|---|
| **58.809.976**<br>Pistas en fact_pista | **142,97 M**<br>Filas en Bronze y Silver | **633 s**<br>Pipeline completo | **116/116**<br>Reglas bloqueantes aprobadas |
| **5.890.921**<br>Lanzamientos | **+0,917**<br>r 1950–2025 | **−0,343**<br>r 2013–2025, no significativa | **2011**<br>Digital supera a óptico |

## Las cinco ideas que debe recordar

- **Solo una tabla estaba recortada.** Silver ya tenía 98,6 M de filas; el tope afectaba a track. El hecho era pequeño por su grano, no por falta de datos.
- **Python no era el problema.** Solo orquestaba: PostgreSQL hacía el trabajo. Decir «Python es lento» es incorrecto.
- **Spark fue una decisión del proyecto.** 58,8 M de filas no es un volumen extremo; un motor de un nodo como DuckDB probablemente alcanzaría. No se hizo ninguna comparación de rendimiento.
- **Los datos completos corrigieron conclusiones.** La caída reciente de duración ya no es significativa y el cruce digital se adelantó de 2012 a 2011.
- **Validar no basta.** Dos errores graves pasaron las primeras reglas porque revisaban integridad, no significado. Se hallaron inspeccionando resultados.

## Cómo usar este documento

| Si necesita | Vaya a |
|---|---|
| Explicar qué se hizo y en qué orden | Sección 3 |
| Mostrar arquitectura y modelo | Secciones 4 y 5 |
| Justificar una tecnología | Sección 7 |
| Explicar errores o demostrar calidad | Secciones 8 y 9 |
| Responder preguntas difíciles | Sección 12 |

Sección 2

# Requisito y punto de partida

El proyecto se entregó en el segundo corte sobre PostgreSQL. El nuevo requisito pide una base de más de 50 millones de registros. Antes de elegir tecnologías se revisó qué se había hecho realmente, porque varias suposiciones iniciales no eran correctas.

| Suposición inicial | Lo que mostró la revisión | Evidencia |
|---|---|---|
| La base se redujo a menos de 20 M | Solo silver.track se limitó a 15.000.000 filas; las demás tablas estaban completas | `TARGET_ROWS` en etl/silver/ingest_tracks_sliced.py |
| Faltaban datos en todo el proyecto | Silver ya sumaba 98.641.617 filas | Volumetría de la entrega anterior |
| El hecho era pequeño por el recorte | Tenía 2.967.681 filas sobre todo por su grano agregado | Grano diario de fact_produccion_musical |
| Python hacía el trabajo y era lento | Python solo orquestaba COPY y SQL; PostgreSQL ejecutaba | ingest_silver.py y scripts SQL |

### Por qué importaba el recorte

Las primeras 15 M líneas del archivo no eran una muestra aleatoria. Además, el corte podía dejar lanzamientos incompletos: el lanzamiento 1438346 aparecía con 21 pistas y 2 soportes, y hoy tiene 33 pistas y 3 soportes. Esa diferencia es consistente con el corte, aunque el volcado también pudo cambiar.

## Recursos disponibles

| Recurso | Valor al iniciar | Consecuencia |
|---|---|---|
| **Memoria RAM** | 13,8 GB | Muy justo para Spark local con este volumen |
| **Procesador** | 16 hilos | Suficiente, pero no resuelve la memoria |
| **Disco libre** | 71 GB | Alcanza para el dump, no para varias capas locales con holgura |
| **Java** | JDK 8 como predeterminado | Spark 4 exige Java 17 [16] |

## Decisiones que tomó el usuario

- Usar Databricks y Spark, y trabajar en la nube en lugar de Spark local.
- Crear los dashboards también en Databricks.
- Corregir las etiquetas y las conclusiones con datos completos.

Sección 3

# Cronología del proceso

El orden fue deliberado: primero comprobar el dato, después medir el riesgo y al final construir. Todo ocurrió entre el 4 y el 5 de noviembre de 2026.

| # | Etapa | Qué se hizo | Resultado |
|---|---|---|---|
| 1 | Diagnóstico | Revisión del ETL y la volumetría anterior | Premisas corregidas (sección 2) |
| 2 | Preparación | CLI de Databricks, inicio de sesión, esquemas y Volume | Workspace listo: landing, bronze, silver, gold, meta |
| 3 | Descarga | Volcado 20261104-002222 de 7,7 GB | SHA-256 correcto |
| 4 | Extracción | 11 tablas, 15 GB, esquema 31 | track con 58.809.976 líneas |
| 5 | Carga | Subida al Volume con la CLI | Falló track por el límite de 5 GiB; se dividió en 8 bloques |
| 6 | Capacidad | Prueba con todas las pistas y joins | Hecho en 93 s; ninguna pista perdida |
| 7 | Conexión | Power BI contra el warehouse con token | 12 filas cargadas: solo prueba conectividad |
| 8 | Pipeline | Bronze, Silver, Gold, validación y job de Lakeflow | Corrida de 633 s con 7 tareas, en estado SUCCESS |
| 9 | Etiquetas | Catálogos oficiales, tarea catalogos y reglas V08 a V11 | Claves y totales idénticos; el job pasó a 8 tareas |
| 10 | Dashboard | 3 páginas en AI/BI | KPI conciliados con Gold |
| 11 | Ajustes visuales | Filtro de año, dispersión, paleta y formato | Revisado en el navegador |
| 12 | Desempate | Regla nueva por país y validación V12 | Corrida de 550 s; Afghanistan de 32.206 a 104 |
| 13 | Publicación | Cinco commits en GitHub | Correo personal retirado del código |

## Prueba de capacidad

Se hizo antes de escribir el pipeline porque era el mayor riesgo: nadie había comprobado que Free Edition soportara el join completo.

| Etapa | Tiempo | Filas |
|---|---|---|
| **Bronze, 9 tablas** | 110 s | 142.841.823 |
| **Hecho con todos los joins** | 93 s | 58.809.976 |
| **Validaciones** | 5 s | 58.809.976 |
| **Agregación por año y formato** | 2 s | 3.230 grupos |

### Qué demuestra y qué no

Esta prueba demuestra que el trabajo termina en Free Edition. No demuestra que sea más rápido que otra opción ni dice cuánta cuota consumió.

Sección 4

# Arquitectura

Se mantuvo la arquitectura Medallón y se agregó Bronze, que no existía. Cada capa tiene una responsabilidad única y puede reconstruirse desde la anterior.




*Figura 1. Flujo de datos, orquestación y gobierno.*

| Capa | Responsabilidad | Regla principal | Tamaño Delta |
|---|---|---|---|
| **Volume** | Archivos originales | No se modifican | 15 GB de TSV |
| **Bronze** | Copia fiel | Todo como texto; conserva escapes | — |
| **Silver** | Datos utilizables | Tipos, escapes, clave única | 4,75 GB |
| **Gold** | Modelo analítico | Estrella, fila cero, aditividad | 1,17 GB |
| **meta** | Gobierno | Registra cargas y reglas | — |

## Tareas del job

| Archivo | Función |
|---|---|
| **00_pruebas.py** | 26 pruebas de reglas de negocio con datos pequeños |
| **01_bronze.py** | Carga el volcado sin interpretar |
| **02_silver.py** | Aplica tipos, revierte escapes y valida claves |
| **02b_catalogos.py** | Carga 6 catálogos oficiales |
| **03_gold_dimensiones.py** | Construye 7 dimensiones con fila cero |
| **04_gold_fact_pista.py** | Construye el hecho a nivel de pista |
| **05_gold_agregados.py** | Construye hechos agregados y tablas mv_* |
| **06_validacion.py** | Integridad, aditividad, catálogos y reconciliación |

- **Reanudable:** cada etapa queda en meta.etl_audit y no se repite para la misma versión del volcado, salvo con el parámetro forzar.
- **Secuencial:** nunca corre más de una tarea a la vez, dentro del límite de 5 tareas concurrentes de Free Edition [1].
- **Falla de forma visible:** si una regla bloqueante no pasa, el job termina en error.

Sección 5

# Modelo de datos

## Decisión de grano

El hecho agregado nunca iba a llegar a 50 M: con todos los datos tiene 11.519.884 filas. Por eso se agregó `fact_pista`, con una fila por pista, y se conservaron los agregados para que el dashboard responda rápido. Así el requisito se cumple sin importar si se interpreta sobre el origen o sobre la tabla de hechos.

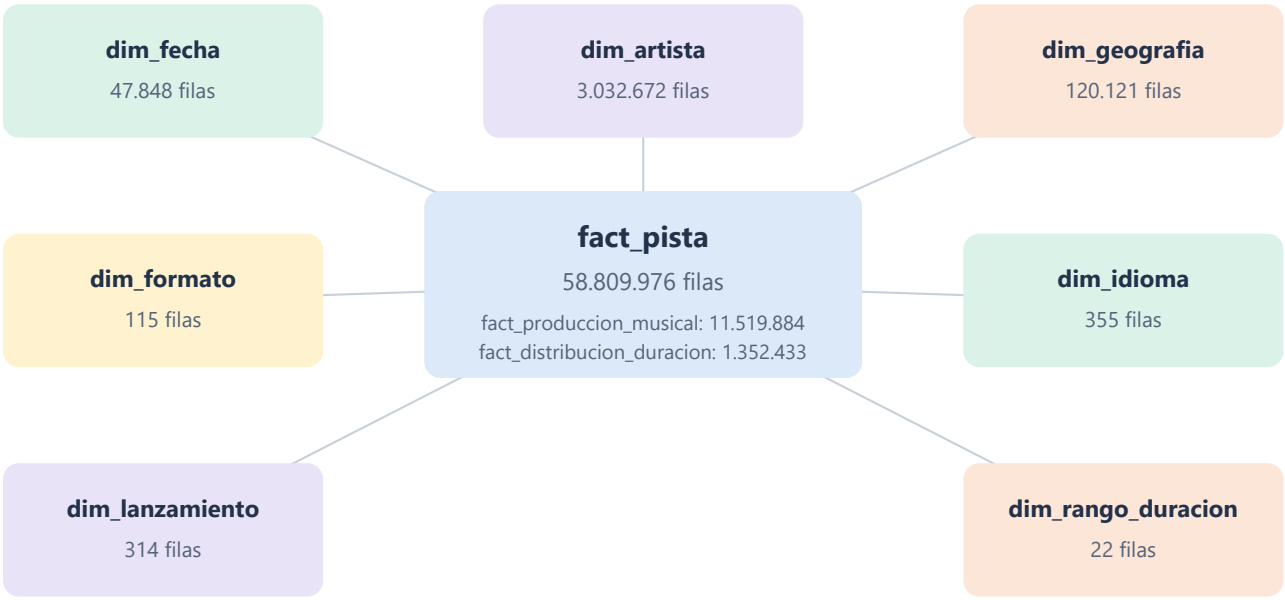



*Figura 2. Hechos de Gold y dimensiones conformadas.*

## Conteos por tabla

| Tabla | Bronze | Silver |
|---|---|---|
| track | 58.809.976 | 58.809.976 |
| recording | 40.826.836 | 40.826.836 |
| release_country | 13.360.406 | 13.360.406 |
| artist_credit_name | 7.364.652 | 7.364.652 |
| isrc | 6.579.238 | 6.579.238 |
| medium | 6.474.432 | 6.474.432 |
| release | 5.903.902 | 5.903.902 |
| artist | 3.032.671 | 3.032.671 |
| release_unknown_country | 489.710 | 489.710 |
| area | 120.120 | 120.120 |
| language | 7.846 | 7.846 |
| Total | 142.969.789 | 142.969.789 |

Silver incluye además 6 catálogos oficiales de hasta 115 filas.

| Tabla Gold | Filas | Tamaño | Clustering |
|---|---|---|---|
| fact_pista | 58.809.976 | 0,77 GB | fecha_key, artista_key |
| fact_produccion_musical | 11.519.884 | 0,22 GB | fecha_key, artista_key |
| fact_distribucion_duracion | 1.352.433 | 11 MB | fecha_key, rango_duracion_key |
| dim_artista | 3.032.672 | 0,12 GB | — |
| dim_geografia | 120.121 | 3 MB | — |
| dim_fecha | 47.848 | < 1 MB | — |
| dim_idioma | 355 | < 1 MB | — |
| dim_lanzamiento | 314 | < 1 MB | — |
| dim_formato | 115 | < 1 MB | — |
| dim_rango_duracion | 22 | < 1 MB | — |
| mv_analitica_artistas | 1.633.826 | 39 MB | artista_key |
| mv_distribucion_geografica | 6.241 | < 1 MB | geografia_key |
| mv_evolucion_formatos | 2.714 | < 1 MB | anio |
| mv_tendencias_produccion_anual | 77 | < 1 MB | anio |

## Reglas de negocio del modelo

| Regla | Definición |
|---|---|
| Fecha | La más antigua del lanzamiento, con prioridad para eventos con país. Años válidos de 1900 a 2030; si no, clave 0. |
| Territorio | País único se conserva. [Worldwide] gana si está entre los empatados. Con 10 o más países, [Worldwide]. Con 2 a 9, el país con más lanzamientos. |
| Artista | El primero del crédito de la pista, o el del lanzamiento. Un artist_credit nunca se usa como artist.id. |
| Duración | La de la pista; si falta, la de la grabación. |
| Primeras pistas | Cada lanzamiento, soporte y grabación suma una vez, en su primera pista. |
| Fila cero | Todo valor faltante apunta a la clave 0: ninguna fila del hecho se pierde. |

| Camino del territorio | Lanzamientos |
|---|---|
| País único | 5.148.387 |
| Sin país (release_unknown_country) | 487.239 |
| 10 o más países: [Worldwide] | 45.043 |
| 2 a 9 países: mayor catálogo | 30.706 |
| [Worldwide] explícito en el empate | 3.223 |

Sección 6

# Reglas de calidad

Hay 116 reglas bloqueantes y 10 indicadores informativos en `meta.data_quality_log`. Las bloqueantes detienen el job; las informativas muestran cuánto dato incompleto existe.

| Grupo | Reglas | Qué comprueba | Tipo |
|---|---|---|---|
| **S01** | 11 | Sin pérdidas al convertir tipos en Silver | Bloqueante |
| **S02** | 11 | Clave primaria única en Silver | Bloqueante |
| **V01** | 22 | Bronze y Silver coinciden con el volcado | Bloqueante |
| **V02** | 3 | Sin huérfanos entre tablas de Silver | Bloqueante |
| **V03** | 14 | Dimensiones con clave única y fila cero | Bloqueante |
| **V04** | 17 | Sin huérfanos en los hechos | Bloqueante |
| **V05** | 6 | Métricas aditivas | Bloqueante |
| **V06** | 2 | Agregados reconcilian con el detalle | Bloqueante |
| **V07** | 1 | Sin medidas negativas | Bloqueante |
| **V08** | 8 | Etiquetas iguales al catálogo oficial | Bloqueante |
| **V09** | 6 | Todos los ID existen en el catálogo | Bloqueante |
| **V10** | 6 | Las traducciones usan nombres oficiales | Bloqueante |
| **V11** | 6 | Catálogo completo | Bloqueante |
| **V12** | 3 | Empates multiterritoriales en [Worldwide] | Bloqueante |
| **I01–I06** | 10 | Pistas sin fecha, artista o geografía; bisiestos; BOM; caminos del territorio | Informativo |

Datos incompletos conocidos: 4.475.092 pistas sin fecha, 1.453.024 sin geografía y 2.716 con años posteriores a 2026.

## Reglas EDA heredadas

| Regla | Tema | En Databricks |
|---|---|---|
| **E01** | Duraciones nulas o en cero | Respaldo con la duración de la grabación |
| **E02** | Fechas parciales o ausentes | Sin imputar: si no hay fecha, clave 0 |
| **E03** | Créditos de artista | Posición 0 de artist_credit_name |
| **E04** | ISRC múltiples | Grabaciones deduplicadas antes del join |
| **E05** | Booleanos en hechos | Banderas en fact_pista; sumas en los agregados |
| **E06** | Textos extensos | STRING de Spark, sin truncar |
| **E07** | Claves huérfanas | Fila cero y regla V04 |
| **E08** | Codificación y BOM | Indicador I05: 1 nombre con BOM |
| **E09** | Cruce geográfico | Unión por identificador entero |
| **E10** | Registro de auditoría | meta.data_quality_log |

Sección 7

# Decisiones de tecnología

Cada registro explica qué se eligió, por qué, qué se descartó y qué costó. La fila **Base** distingue lo **medido en este proyecto** de lo que viene de **documentación** y de lo que es **criterio**.

### Límite de esta comparación

**No se ejecutó ningún benchmark comparativo.** PostgreSQL y Spark no se midieron con los mismos datos. 58,8 M de filas no es un volumen extremo, y un motor de un nodo como DuckDB probablemente podría con él. Las comparaciones siguientes son cualitativas, basadas en documentación y en las condiciones del proyecto. Las condiciones comerciales cambian: se consultaron el 5 de noviembre de 2026.

## ADR-01. Plataforma

| | |
|---|---|
| **Decisión** | Databricks con Apache Spark |
| **Por qué** | Es la plataforma solicitada para el proyecto. Da Spark administrado, almacenamiento Delta, catálogo, orquestación y dashboards en un solo lugar, sin operar infraestructura. |
| **Costo real** | Dependencia de una plataforma de terceros y curva de aprendizaje. El dashboard y la orquestación no se trasladan directamente a otra herramienta. |
| **Base** | **Medido:** la carga completa termina en 633 s. **Criterio:** elección de plataforma. **Documentación:** condiciones de alternativas. |

| Alternativa | Por qué no se eligió |
|---|---|
| **PostgreSQL local** | Ya existía y sigue siendo válido para este tamaño. Se descartó porque el proyecto buscaba procesamiento distribuido y por el disco disponible. |
| **Spark local** | 13,8 GB de RAM y JDK 8. Spark 4 exige Java 17 [16]; además, un solo equipo no muestra procesamiento distribuido. |
| **DuckDB** | Probablemente resolvería la carga en un nodo, con procesamiento mayor que la memoria y límites conocidos [11]. No cumple el objetivo de usar Spark. |
| **Snowflake** | Prueba limitada a 30 días o 400 USD [13]. Es un almacén SQL; no era el objetivo de Spark. |
| **BigQuery** | El sandbox da 10 GB de almacenamiento y las tablas expiran [12]; los 15 GB de archivos superan ese margen. |
| **Amazon Redshift** | Crédito de 300 USD por 90 días [14]; exige una cuenta de AWS y no usa Spark como motor. |
| **Microsoft Fabric** | Prueba de 60 días [15]. Es viable, pero no era la plataforma pedida y la prueba caduca. |
| **Hadoop y Hive** | Exigen operar un clúster y su enfoque por lotes es más pesado [17]. Sin ventaja para este caso. |

## ADR-02. Plan de Databricks

| | |
|---|---|
| **Decisión** | Free Edition |
| **Por qué** | No tiene costo y alcanza para el proyecto: se comprobó con la prueba de capacidad y con el pipeline completo. |
| **Costo real** | Solo cómputo serverless, un warehouse 2X-Small, cuota diaria que puede apagar el cómputo, sin SLA y sin uso comercial [1]. La cuota consumida es desconocida. |
| **Base** | **Medido:** todas las corridas terminaron. **Sin medir:** cuota usada. **Documentación:** límites del plan. |

| Alternativa | Por qué no se eligió |
|---|---|
| **Plan pago o prueba** | No fue necesario. Quedaba como respaldo si la cuota no alcanzaba. |

## ADR-03. Lenguaje

| | |
|---|---|
| **Decisión** | PySpark para el pipeline y Spark SQL para consultas |
| **Por qué** | Reutiliza la lógica SQL existente, permite probar funciones con datos pequeños y es el lenguaje disponible en este entorno. |
| **Costo real** | Hay que respetar restricciones de serverless: sin caché de DataFrame y con configuración de Spark limitada [2]. |
| **Base** | **Documentación:** soporte de lenguajes. **Criterio:** reutilizar SQL. |

| Alternativa | Por qué no se eligió |
|---|---|
| **Scala** | No está soportado en Free Edition ni en serverless [1, 2]. |
| **pandas** | Trabaja en la memoria de un solo proceso; no es adecuado para 58,8 M de filas con varios joins. |

## ADR-04. Formato de tablas

| | |
|---|---|
| **Decisión** | Delta Lake |
| **Por qué** | Las tablas administradas de Unity Catalog usan Delta por defecto [4]. Aporta transacciones e historial. El historial se usó para comparar versiones y probar que las claves no cambiaron al corregir etiquetas. |
| **Costo real** | Las tablas se leen mejor dentro de herramientas compatibles con Delta. |
| **Base** | **Medido:** comparación por *time travel* sin diferencias en claves. **Documentación:** soporte de formatos. |

| Alternativa | Por qué no se eligió |
|---|---|
| **Parquet sin Delta** | Sin transacciones ni historial; una escritura fallida podría dejar datos a medias. |
| **Apache Iceberg** | Es abierto y está soportado [4], y UniForm permite leer Delta como Iceberg [10]. No había otro motor que justificara el cambio. |
| **Apache Hudi** | Orientado a actualizaciones incrementales, que este proyecto no necesita: cada carga es un volcado completo. |

## ADR-05. Archivos crudos

| | |
|---|---|
| **Decisión** | Volume administrado de Unity Catalog |
| **Por qué** | Gobierna archivos igual que tablas. Databricks marca como obsoletos DBFS root y los montajes, y recomienda volúmenes [3]. |
| **Costo real** | Cada archivo subido por la CLI tuvo un tope de 5 GiB, y por eso track se dividió. |
| **Base** | **Medido:** error real de la CLI. **Documentación:** recomendación oficial. |

| Alternativa | Por qué no se eligió |
|---|---|
| **DBFS root o montajes** | Obsoletos y no recomendados [3]. |
| **Almacenamiento externo propio** | Exige una cuenta de nube y credenciales; Free Edition no admite ubicaciones personalizadas [1]. |

## ADR-06. Orquestación

| | |
|---|---|
| **Decisión** | Lakeflow Jobs |
| **Por qué** | Encadena tareas con dependencias, registra cada corrida y permite ejecutar solo una parte. Los Jobs son procedimentales, lo que encaja con notebooks y pruebas [6]. |
| **Costo real** | La definición del job es específica de Databricks. |
| **Base** | **Medido:** corridas 1040003573052070 y 853384881134750 en SUCCESS. **Documentación:** diferencia entre Jobs y pipelines. |

| Alternativa | Por qué no se eligió |
|---|---|
| **Apache Airflow** | Requiere instalar y mantener otro servicio, sin beneficio para un flujo lineal. |
| **Declarative Pipelines** | Definen transformaciones de forma declarativa [6]. Exigían reescribir la lógica y Free Edition limita los pipelines activos [1]. |
| **Script en Python** | Era la opción anterior: sin dependencias administradas ni historial de corridas. |

## ADR-07. Organización física

| | |
|---|---|
| **Decisión** | Liquid clustering y tablas agregadas |
| **Por qué** | Databricks lo presenta como reemplazo de particiones y ZORDER [5]. Evita fijar particiones sobre datos desiguales: la producción se concentra en las décadas recientes y 7,6 % de las pistas no tiene fecha. |
| **Costo real** | **Su beneficio no se midió.** No se comparó contra una tabla sin clustering; los tiempos actuales no se le pueden atribuir. |
| **Base** | **Medido:** consultas de 1,3 a 3,1 s en 2X-Small, sin comparación. **Documentación:** recomendación oficial. |

| Alternativa | Por qué no se eligió |
|---|---|
| **Particiones por rango** | Era el diseño en PostgreSQL. En Delta exige una elección fija y puede crear demasiados archivos pequeños. |
| **ZORDER** | Requiere reoptimizar manualmente y Databricks recomienda liquid clustering [5]. |

## ADR-08. Cómputo

| | |
|---|---|
| **Decisión** | Serverless |
| **Por qué** | Es el único tipo disponible en Free Edition [1]. Arranca sin configurar clústeres. |
| **Costo real** | Sin Spark UI, caché de DataFrame ni la mayoría de configuraciones [2]. Se usa el perfil de consultas para diagnosticar. |
| **Base** | **Documentación:** limitaciones de serverless. |

| Alternativa | Por qué no se eligió |
|---|---|
| **Clústeres clásicos** | No están disponibles en este plan. |

## ADR-09. Visualización

| | |
|---|---|
| **Decisión** | Dashboard AI/BI como único dashboard vigente |
| **Por qué** | AI/BI corre sobre el warehouse y evita importar 11,5 M de filas a un equipo con 13,8 GB de RAM. Con el dashboard ya publicado y conciliado, mantener otro reporte no aportaba a esta entrega. |
| **Costo real** | El drill-down funciona dentro de tablas dinámicas, no en los gráficos, y llega a mes. Top N no se recalcula con filtros. |
| **Base** | **Medido:** filtros iguales a SQL. **Sin medir:** Power BI con 11,5 M de filas, porque no se continuó por esa vía. |

| Alternativa | Por qué no se eligió |
|---|---|
| **Reconectar Power BI** | Riesgo de memoria sin comprobar: la conexión solo se probó con 12 filas. Se decidió no continuar. |
| **Mantener ambos** | Dos herramientas que mantener y dos versiones de las mismas cifras. |

## ADR-10. Cálculo de primeras pistas

| | |
|---|---|
| **Decisión** | `min(struct(...))` con agregación |
| **Por qué** | Encuentra la primera pista por lanzamiento, soporte y grabación sin ordenar particiones completas, como hacen las funciones de ventana. |
| **Costo real** | Es menos evidente al leerlo. Una prueba con datos pequeños documenta el comportamiento. |
| **Base** | **Criterio técnico:** no se midió contra ROW_NUMBER. **Medido:** la aditividad coincide. |

| Alternativa | Por qué no se eligió |
|---|---|
| `ROW_NUMBER` | Es más directo y era la lógica anterior, pero requiere tres ordenamientos sobre 58,8 M de filas. |

## ADR-11. Carga de archivos

| | |
|---|---|
| **Decisión** | CLI y división de track en bloques de 1 GB |
| **Por qué** | Automatizable y repetible. Dividir por líneas evita cortar registros, y Spark lee la carpeta como una tabla. |
| **Costo real** | La suposición inicial de que la CLI admitía más de 5 GB fue incorrecta y costó una subida fallida. |
| **Base** | **Medido:** 8 bloques con los mismos bytes y 58.809.976 líneas. |

| Alternativa | Por qué no se eligió |
|---|---|
| **Interfaz web** | Límite de 5 GB por archivo [7] y sin repetibilidad. |
| **SDK o API** | Podría evitar el límite [7], pero requería más código para una carga única. |
| **Comprimir con gzip** | No se puede dividir para procesar en paralelo. |

## ADR-12. Etiquetas de dimensiones

| | |
|---|---|
| **Decisión** | Catálogos oficiales del volcado |
| **Por qué** | Traducir por nombre oficial evita errores por ID. Las reglas V08 a V11 bloquean diferencias. |
| **Costo real** | La calidad no tiene catálogo en el volcado: sus nombres y la asignación de categorías de formato son criterio propio. 5.875.755 lanzamientos quedan «Sin calificar». |
| **Base** | **Medido:** la regla falló contra las dimensiones viejas y pasó contra las nuevas. |

| Alternativa | Por qué no se eligió |
|---|---|
| **Mapas por ID en el código** | Fue la causa del error: 76.524 ciudades aparecían como Condado. |

Sección 8

# Errores encontrados y correcciones

Documentar los errores es parte de la defensa: muestra cómo se verificó el resultado. La columna **Detección** es la más importante.

| Error | Impacto | Corrección | Detección |
|---|---|---|---|
| **Conteos no aditivos (entrega anterior)** | KPI principal inflado | Asignación a la primera pista | Conciliación |
| **Orden de columnas de medium** | edits_pending y track_count invertidos en el DDL anterior | Orden real del volcado | Inspección de archivos |
| **Etiquetas contra catálogos** | 5.875.755 lanzamientos como «Baja»; 81.904 áreas mal rotuladas | Catálogos oficiales y reglas V08 a V11 | **Inspección**; pasó la primera validación |
| **Desempate por menor ID (Afghanistan)** | 32.206 lanzamientos asignados a Afghanistan | Regla nueva y V12 | **Inspección**; pasó la primera validación |
| **Límite de 5 GiB** | track no subió | 8 bloques de 1 GB | Error de la CLI |
| **Falso positivo en V12.grano** | Marcó 51.729 casos válidos | Regla acotada a eventos con país | Corrida en rojo |
| **Año 0 en el filtro** | El rango empezaba en 0 | Filtro limitado a años reales | Revisión del usuario |
| **Gráfico de dispersión** | No cargaba | Cálculo en un dataset propio | Revisión del usuario |
| **Versión de tabla inválida** | Widget con error | Se restauró la versión 2 | Revisión en el navegador |
| **Caché del dashboard** | Mostraba datos anteriores a la corrección | Actualización forzada | Revisión en el navegador |

### Por qué pasó la primera validación

Afghanistan tiene el ID de área 1. En un empate de fecha, `min(struct(...))` terminaba comparando el ID de país y ganaba el menor. Por eso reunía 32.206 lanzamientos de 23.640 artistas, y solo 32 lanzamientos eran de artistas de ese país. Tras la corrección quedó en 104. Se movieron 639.777 pistas de área y ninguna cambió de fecha.

## Lección

Las reglas iniciales comprobaban integridad: claves válidas, sumas correctas. Un dato puede ser íntegro y estar mal. Después de cada hallazgo se agregó una regla bloqueante, así que esos errores ya no pueden volver sin detener el job.

Sección 9

# Evidencia de validación

| 26/26 | 94 | 22 | 2 |
|---|---|---|---|
| Pruebas con datos pequeños | Reglas de validación final | Reglas de Silver | Filtros comparados con SQL |

## Pruebas

Corren dentro de Databricks con datos pequeños, porque el entorno local no tiene PySpark. Cada grupo se vio fallar primero y después pasar. Son la primera tarea del job: si fallan, no se procesa nada.

## Reconciliación

| Comprobación | Valor | Resultado |
|---|---|---|
| **Pistas: volcado = Bronze = Silver = fact_pista** | 58.809.976 | Coincide |
| **Lanzamientos: suma = conteo distinto** | 5.890.921 | Coincide |
| **Soportes: suma = conteo distinto** | 6.463.097 | Coincide |
| **Grabaciones: suma = conteo distinto** | 40.669.195 | Coincide |
| **Pistas con duración: distribución = agregado** | 54.468.610 | Coincide |

## Corridas

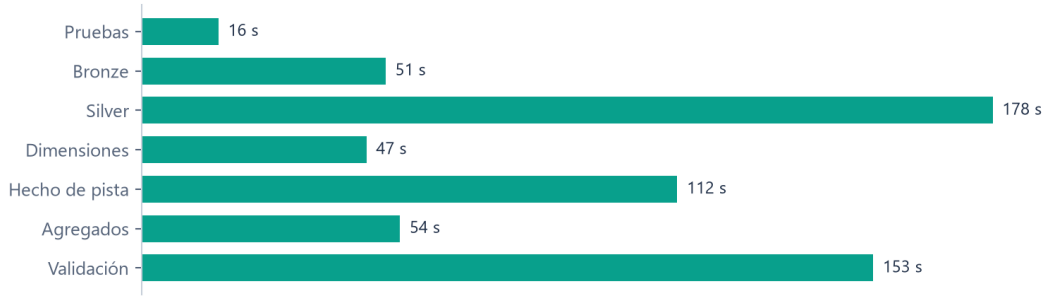



*Figura 3. Tareas de la corrida completa 1040003573052070.*

| Corrida | Alcance | Duración | Estado |
|---|---|---|---|
| **1040003573052070** | Pipeline completo; Bronze reutilizó 9 tablas de la prueba | 633 s | SUCCESS |
| **360438219893932** | Dimensiones, agregados y validación tras corregir etiquetas | 262 s | SUCCESS |
| **853384881134750** | Pruebas, hecho, agregados y validación tras el desempate | 550 s | SUCCESS |

## Filtros del dashboard contra SQL

| Filtro | Lanzamientos o razón | Pistas o duración | Tercer valor | Coincidencia |
|---|---|---|---|---|
| **Año 2000–2010** | 1.169.839 | 15.574.369 | 599.256 artistas | Exacta |
| **Década 1980s** | 9,2 pistas por lanzamiento | 4,07 min | 132.202 horas | Exacta |

Se probaron estos dos filtros; los demás filtros de lista usan el mismo control, pero no se probaron uno por uno.

Sección 10

# Resultados

| Métrica | Carga de 15 M | Volcado completo |
|---|---|---|
| Pistas | 15.000.000 | 58.809.976 |
| Lanzamientos | 1.728.603 | 5.890.921 |
| Soportes | 1.882.045 | 6.463.097 |
| Filas del hecho agregado | 2.967.681 | 11.519.884 |
| r año-duración 1950–2025 | +0,89 | +0,917 |
| r año-duración 2013–2025 | −0,57 | −0,343 |
| Digital supera a óptico | 2012 | 2011 |

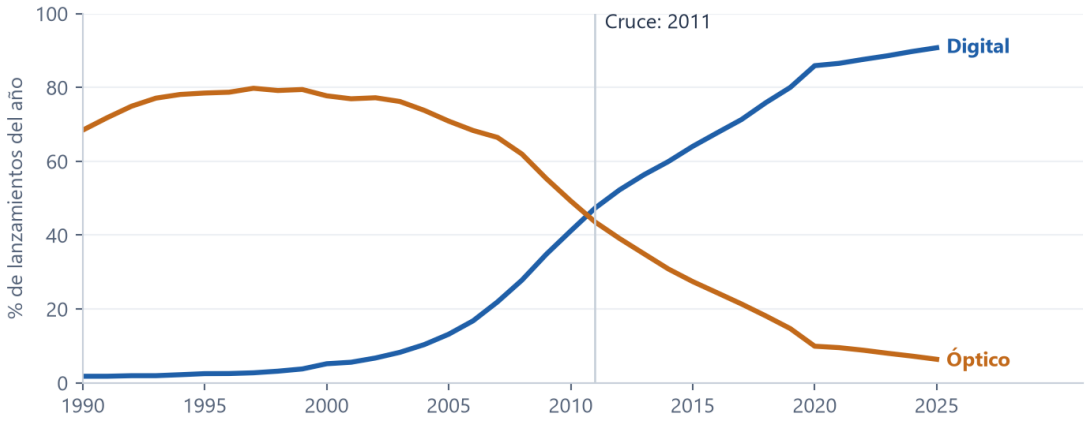



*Figura 4. Digital supera a óptico en 2011.*

- **Formatos:** en 2025, digital 90,7 % y óptico 6,4 % de los lanzamientos con fecha.
- **Duración:** media de 4:14; las bandas de 3:00 a 4:00 reúnen 28,6 % de las pistas.
- **RF05:** r = +0,917 (1950–2025), +0,858 (1990–2025) y −0,343 (2013–2025). El último valor, con n = 13, no supera el valor crítico de 0,553.
- **ISRC:** 29,1 % de las pistas en este catálogo.
- **Territorio:** [Worldwide] reúne 2.098.419 lanzamientos; el primer país, United States, 976.129.

Sección 11

# Limitaciones conocidas

| Tema | Limitación | Impacto |
|---|---|---|
| **Dashboard** | Drill-down en tablas dinámicas, hasta mes y no hasta día | RF03 queda cubierto sin el nivel día |
| **Dashboard** | Clic para filtrar solo entre gráficos del mismo dataset | Otros datasets no reaccionan |
| **Modelo** | Las ciudades no están enlazadas con su país | El drill geográfico tiene poca profundidad |
| **Dashboard** | Top N no se recalcula con filtros | Los rankings no cambian de integrantes al filtrar |
| **Dashboard** | Tablas sin separador de miles | Solo visual; se probaron tres formatos |
| **Dashboard** | Tintes de Mezcla de formatos fuera del umbral de la paleta | Se distinguen por leyenda y posición |
| **Datos** | Fechas anteriores a 1900 pasan a «sin fecha» | 1.266 registros de país, incluido un año 206 |
| **Datos** | 2.716 pistas con años posteriores a 2026 | Las series terminan en 2025 |
| **Datos** | Nombres de calidad según conocimiento de MusicBrainz | El volcado no los trae como catálogo |
| **Datos** | Categoría de formato según criterio propio | Afecta la lectura de digital frente a físico |
| **Datos** | Fila 0 de dim_lanzamiento con 770 lanzamientos reales | No es un desconocido puro |
| **Datos** | Regla de 10 países definida por el proyecto | Otro umbral movería lanzamientos |
| **Power BI** | El reporte queda con la carga de 15 M de la entrega anterior | Sus cifras no son las vigentes; el dashboard oficial es AI/BI |
| **Rendimiento** | Sin comparación entre motores | No se puede afirmar que Spark sea más rápido aquí |
| **Rendimiento** | Beneficio del liquid clustering sin medir | No se le pueden atribuir los tiempos |
| **Rendimiento** | Cuota de Free Edition desconocida | Una corrida futura podría interrumpirse |
| **Rendimiento** | Varios usuarios sin probar | El warehouse es 2X-Small |

Sección 12

# Guía de defensa

Respuestas cortas y honestas. Si no conoce un dato, diga qué se midió y qué no: es mejor que improvisar.

## Preguntas difíciles

**P: ¿Por qué Spark si 58 M de filas caben en una sola máquina?**

**R:** Porque el objetivo del proyecto era trabajar con procesamiento distribuido y arquitectura lakehouse. Es cierto que un motor de un nodo como DuckDB probablemente alcanzaría. Spark no se eligió por necesidad de volumen.

**P: ¿Por qué no hicieron un benchmark real?**

**R:** Porque la prioridad fue cumplir el requisito y validar los datos. Solo medimos que el proceso termina y cuánto tarda. No afirmamos que Spark sea más rápido que PostgreSQL. Es trabajo futuro.

**P: ¿Por qué ya no usan Power BI?**

**R:** AI/BI corre en la nube y evita importar 11,5 M de filas a un equipo limitado. El drill-down se conserva con tablas dinámicas, y un clic en una barra filtra los gráficos del mismo dataset. El reporte de Power BI queda como parte de la entrega anterior.

**P: Si Python no era lento, ¿qué ganaron?**

**R:** Dejamos un motor de un nodo con disco limitado y pasamos a uno distribuido administrado, con historial, catálogo y orquestación. La ganancia demostrada es funcional, no de velocidad comparada.

**P: ¿Cuánto cuesta esto en producción?**

**R:** No lo sabemos. Free Edition no sirve para uso comercial y no medimos la cuota consumida.

**P: ¿Sus reglas garantizan que los datos están bien?**

**R:** No. Garantizan lo que comprueban. Dos errores pasaron las primeras reglas; por eso también revisamos resultados.

## Modelo y datos

**P: ¿Dónde están los 50 M de registros?**

**R:** En 58.809.976 filas de fact_pista, y 142.969.789 filas en Silver.

**P: ¿Por qué conservar hechos agregados?**

**R:** Porque el dashboard no necesita detalle por pista y responde más rápido con menos filas.

**P: ¿Cómo evitan el doble conteo?**

**R:** Cada lanzamiento suma solo en su primera pista. La suma da 5.890.921, igual al conteo distinto.

**P: ¿Por qué 5.890.921 y no 5.903.902 lanzamientos?**

**R:** Porque 12.981 lanzamientos no tienen pistas en el volcado.

**P: ¿Por qué hay tantos lanzamientos «Sin calificar»?**

**R:** Porque −1 es el valor por defecto en MusicBrainz. No significa baja calidad.

**P: ¿Por qué [Worldwide] supera a Estados Unidos?**

**R:** Reúne lanzamientos publicados en muchos territorios. Se muestra aparte porque no es un país.

## Tecnología

**P: ¿Qué aporta Delta sobre Parquet?**

**R:** Transacciones e historial. Usamos el historial para probar que las claves no cambiaron.

**P: ¿Qué significa serverless aquí?**

**R:** Databricks administra el cómputo: no configuramos clústeres. Es lo único disponible en Free Edition.

**P: ¿Liquid clustering mejoró el rendimiento?**

**R:** No lo sabemos: no lo comparamos contra una tabla sin clustering.

**P: ¿Por qué no Scala?**

**R:** No está soportado en Free Edition ni en serverless.

## Resultados

**P: ¿El streaming acortó las canciones?**

**R:** Los datos no permiten afirmarlo. Entre 2013 y 2025, r = −0,343 con 13 años, no significativa, y Pearson no prueba causalidad.

**P: ¿Por qué cambiaron sus conclusiones?**

**R:** Porque la carga anterior era un subconjunto por orden de archivo. Con todos los datos corregimos lo que no se sostenía.

**P: ¿Digital superó al CD en 2011?**

**R:** Superó a la categoría óptica, que incluye CD, DVD y similares. Es más preciso decirlo así.

Sección 13

# Reproducción y mantenimiento

Los pasos detallados están en `databricks/README.md`. Este es el recorrido completo.

| # | Paso | Detalle |
|---|---|---|
| 1 | Preparar el workspace | Crear cuenta, instalar la CLI e iniciar sesión con un perfil propio |
| 2 | Crear esquemas | landing, bronze, silver, gold y meta, más el Volume mbdump |
| 3 | Descargar y verificar | Volcado completo y comprobación SHA-256 [18] |
| 4 | Subir archivos | Dividir track en bloques y copiar con la CLI |
| 5 | Importar el código | Subir databricks/pipeline al workspace |
| 6 | Crear el job | Reemplazar el marcador de usuario en una copia local de job.json |
| 7 | Ejecutar y validar | Correr el job y revisar meta.data_quality_log |
| 8 | Desplegar el dashboard | Crear y publicar desde el archivo .lvdash.json |
| 9 | Actualizar el dashboard | Forzar actualización después de reconstruir Gold |
| 10 | Regenerar documentos | Ejecutar generar_metricas_databricks.py y los scripts de documentos |

**Seguridad**

No guarde tokens ni datos personales en el repositorio: es público. El archivo job.json usa un marcador y la copia local con su usuario no se agrega a Git.

## Archivos principales

| Ruta | Contenido |
|---|---|
| **databricks/pipeline/** | Pipeline, pruebas, validación y definición del job |
| **databricks/dashboards/** | Definición del dashboard AI/BI |
| **databricks/spikes/** | Prueba de capacidad descartable |
| **etl/, sql/, powerbi/** | Entrega anterior sobre PostgreSQL |
| **scripts/generar_metricas_databricks.py** | Consulta cifras y genera docs/build/metricas_databricks.json |
| **scripts/build_*.py y build_sustentacion_v4.js** | Generan la presentación y los PDF |

Sección 14

# Referencias

Consultadas el 5 de noviembre de 2026. Las condiciones de pruebas y planes pueden cambiar. La referencia 17 es una fuente secundaria.


[1] Databricks. Free Edition limitations. https://docs.databricks.com/aws/en/getting-started/free-edition-limitations

[2] Databricks. Serverless compute limitations. https://docs.databricks.com/aws/en/compute/serverless/limitations

[3] Databricks. Best practices for DBFS and Unity Catalog. https://docs.databricks.com/en/dbfs/unity-catalog.html

[4] Databricks. Tables concepts. https://docs.databricks.com/aws/tables

[5] Databricks. Use liquid clustering for tables. https://docs.databricks.com/aws/en/delta/clustering

[6] Databricks. Pipeline task for jobs. https://docs.databricks.com/en/jobs/pipeline.html

[7] Databricks. Upload files to a volume. https://docs.databricks.com/en/ingestion/add-data/upload-to-volume.html

[8] Databricks. Dashboard concepts. https://docs.databricks.com/aws/en/dashboards/concepts

[9] Databricks. Dashboard limits. https://docs.databricks.com/aws/en/dashboards/limits

[10] Databricks. Delta UniForm: universal format for lakehouse interoperability. https://databricks.com/blog/delta-uniform-universal-format-lakehouse-interoperability

[11] DuckDB. Tuning workloads: larger-than-memory workloads. https://duckdb.org/docs/current/guides/performance/how_to_tune_workloads

[12] Google Cloud. BigQuery sandbox. https://cloud.google.com/bigquery/docs/sandbox

[13] Snowflake. Página de prueba gratuita. https://signup.snowflake.com

[14] Amazon Web Services. Amazon Redshift free trial. https://aws.amazon.com/redshift/free-trial

[15] Microsoft. Fabric trial. https://learn.microsoft.com/fabric/get-started/fabric-trial

[16] Apache Spark. SPARK-46910: Java mínimo para Spark 4. https://issues.apache.org/jira/browse/SPARK-46910

[17] Logz.io. Comparing Apache Hive vs. Spark (fuente secundaria). https://logz.io/blog/hive-vs-spark/

[18] MetaBrainz. Volcados completos de MusicBrainz. https://data.metabrainz.org/pub/musicbrainz/data/fullexport/